TEAM REFERENCE GUIDE

Presentation Guidebook

WiDS Global Datathon 2026 — Wildfire Survival Prediction

DeMarcus Crump · Chloe Tu · Akinbobola Akinpelu · Aima Ayaz
Houston City College | ITAI 2377 Data Science in Artificial Intelligence | Spring 2026

How to use this guide: Read the project overview first. Then use the Glossary to get comfortable with terminology. Then read the Q&A section — those are the exact types of questions your instructor will likely ask. The analogy boxes throughout will help you explain things simply if asked.

PROJECT OVERVIEW

What Is This Project, in Plain English?

This was a **Kaggle data science competition** run by WiDS (Women in Data Science) for their 2026 Global Datathon. The challenge: given sensor and environmental data about an active wildfire, predict whether that fire will **reach a community evacuation zone** — and if so, within how many hours (12, 24, 48, or 72).

Our class participated as part of **ITAI 2377** at Houston City College. This was a real competition with real data, not a made-up classroom exercise. Teams submitted predictions to Kaggle's leaderboard.

THE DATA WE HAD

221 fire trajectories — records of past wildfires, each with weather readings (wind speed, humidity, temperature), satellite measurements, and whether the fire ultimately reached the community zone. That's a small dataset by data science standards. Only about **31%** of those fires actually reached the zone — the rest were contained beforehand.

SIMPLE ANALOGY

Imagine you're a doctor tracking 221 patients to see if their illness gets severe enough to need surgery. Some patients recovered on their own before you found out either way. You can't just throw away those patients — their recoveries still tell you something useful. Our wildfire data had the same structure: fires that were contained before reaching the zone still told us what conditions slow a fire down.

THE CORE DECISION

Why Survival Analysis — and Not Something Simpler?

The obvious first instinct was to treat this as a **classification problem**: will the fire reach the zone, yes or no? Standard tools — logistic regression, random forests, even neural networks — all handle that. We considered them all. The problem: with only 221 training examples, a powerful model would simply *memorize* the training data rather than learn anything that would work on a fire it had never seen before. This is called **overfitting**.

We also had a more subtle problem: the **incomplete outcomes**. About 69% of fires in our dataset were contained before reaching the zone. A standard classifier sees those as "did not reach zone" — clean negatives. But that's not quite right. A fire contained at hour 10 is different from one contained at hour 70. The timing matters, and a standard model throws that information away.

Survival analysis was designed specifically for this situation. Originally developed in medicine, it answers the question "not just if, but *when*." It also has a formal way to handle those fires that were contained before an outcome — they're called **right-censored** observations, and survival analysis uses them rather than discards them.

ANALOGY FOR THE PROFESSOR

Think of it like a marathon. A standard classifier asks "did the runner finish?" Survival analysis asks "how long did it take them to finish — and for runners who dropped out, how far did they get before quitting?" The dropout data still tells you something about the course difficulty. That's exactly what right-censored wildfire data gives us.

THE TWO MODELS WE USED

We used two survival analysis algorithms and combined their outputs: **Random Survival Forest (RSF)** — a tree-based model that handles complex nonlinear relationships well, and **Gradient Boosting Survival Analysis (GBSA)** — a boosting model that learns from its own errors sequentially. Combining them (an **ensemble**) gave more stable, reliable predictions than either alone.

FEATURE ENGINEERING

Why We Didn't Just Feed Raw Data Into the Model

The dataset came with raw sensor columns: wind speed, temperature, humidity, fire perimeter coordinates, satellite readings. We could have fed those directly into the model. Instead, we engineered new features — calculated columns that described *what the fire was actually doing* in physical terms.

The Three Key Engineered Features

1. DANGER INDEX

A composite score combining wind speed, low humidity, and high temperature into a single number representing how dangerous current conditions were for fire spread. Rather than giving the model three separate columns it has to figure out how to combine, we did the combining upfront based on what fire science actually says.

2. TRAJECTORY ALIGNMENT

A measure of how directly the fire was moving *toward* the community zone. A fire moving at high speed in the wrong direction is less dangerous than a slower fire pointed directly at the evacuation zone. This feature captured that directional relationship.

3. TIME-TO-IMPACT ESTIMATE

An estimated arrival time calculated from the fire's current speed and its distance from the zone. Essentially a physics-based "if nothing changes, this fire arrives in X hours" estimate.

ANALOGY

Instead of giving a navigator raw GPS coordinates every second, you hand them a processed summary: "you're 20 miles out, heading directly toward the destination, at 60 mph — ETA 20 minutes." The raw coordinates are still technically in there, but the engineered summary is what the navigator actually needs to make a decision. Our model was the navigator.

VALIDATION & HONESTY

How We Made Sure Our Results Were Real

Getting a high score is easy if you cheat — intentionally or accidentally. In a competition, teams can tune their model specifically to the public leaderboard examples, which inflates their score without making a better model. We wanted to avoid that.

We used **Stratified K-Fold cross-validation**. Here's what that means: we split our 221 fire records into 5 groups (folds). We trained the model on 4 groups and tested on the 1 remaining — then rotated which group was the test set, repeating 5 times. *Stratified* means each fold was carefully balanced to contain roughly 31% "reached zone" fires, matching the real distribution of the data.

The score we report — **0.9737** — is an average across all 5 test folds. It was never trained on that data. That's an honest number.

ANALOGY

Imagine studying for a test using practice exams. Bad study habit: memorize the exact answers to the practice tests. Good study habit: cover the answers, test yourself on different problems, and check your score honestly. Our validation was the second approach. Some leaderboard teams were doing the first.

WHAT DOES 0.9737 MEAN?

Our metric is called the **concordance index (C-index)**. A score of 0.5 means the model is guessing randomly. A score of 1.0 is perfect. Our 0.9737 means that in 97.37% of paired comparisons — "which of these two fires reached the zone sooner?" — our model got it right. For a dataset of 221 records, that's a strong, defensible result.

REFERENCE

Glossary — Terms You Need to Know

WiDS Datathon / What is it?

Women in Data Science Global Datathon — an annual Kaggle-hosted data science competition open to teams worldwide. Our class participated as part of ITAI 2377. It's a real competition, not a classroom simulation.

Kaggle

The world's largest data science competition platform, owned by Google. Teams upload predictions and get scored on a public leaderboard. Think of it as the Olympics for data scientists.

Survival Analysis

A branch of statistics for modeling *time-to-event* data. Instead of asking "will X happen?" it asks "when will X happen?" Originally developed in medical research (e.g., how long until a patient recovers or dies). We applied it to wildfires to predict when a fire would reach the evacuation zone.

Right-Censored Data

An observation where the event of interest hadn't happened yet when the study ended. In our case: fires that were contained before reaching the zone. We know they *hadn't* reached the zone up to a certain time, but we don't know what would have happened after. Survival analysis uses these partial observations rather than discarding them.

Overfitting

When a model learns the training data so well that it memorizes noise and quirks specific to those examples, and fails on new data it hasn't seen. The bigger the model, the smaller the dataset, the higher the overfitting risk. With only 221 records, this was our primary concern.

Random Survival Forest (RSF)

A survival analysis version of the Random Forest algorithm. Builds many decision trees on random subsets of the data, then combines their predictions. Good at capturing complex, nonlinear relationships in small datasets.

Gradient Boosting Survival Analysis (GBSA)

A survival analysis version of gradient boosting. Builds a sequence of simple models where each one corrects the errors of the previous one. Complementary strengths to RSF, which is why we used both.

Ensemble Model

Combining the predictions of two or more models into one final prediction. Usually more accurate and more stable than any single model alone. We weighted RSF and GBSA outputs together to produce our final predictions.

Feature Engineering

The process of creating new input columns by transforming or combining raw data. Rather than feeding raw sensor numbers to the model, we computed physics-based summaries (danger index, trajectory alignment, time-to-impact) that described what the fire was actually doing.

Stratified K-Fold Cross-Validation

A method for honestly evaluating a model. Split the data into K groups (we used 5). Train on K-1 groups, test on the remaining one. Rotate K times. Average your scores. *Stratified* means each fold maintains the same ratio of outcomes (31% fires reached zone) as the full dataset. This prevents accidentally testing on easy or hard subsets.

Concordance Index (C-index)

The performance metric used in survival analysis. It measures how often the model correctly ranks which of two events happened first. A score of 0.5 is random guessing; 1.0 is perfect. Our score was 0.9737, meaning the model correctly ranked 97.37% of fire pairs by time-to-impact.

Bayesian Hyperparameter Tuning (Optuna)

A smart way to find the best model settings. Instead of trying every possible combination (slow), Bayesian tuning learns from previous attempts and focuses on the most promising regions of the settings space. We used a library called Optuna to automate this.

Monotonic Probability Bounds

A constraint we applied to our output predictions to make them physically sensible: the probability of a fire reaching the zone within 48 hours must be at least as high as the probability within 24 hours (since 48 hours includes more time). Without this constraint, the raw model outputs could produce illogical results.

Hazard Function

In survival analysis, the hazard function represents the instantaneous rate of an event happening at a given time, given it hasn't happened yet. Informally: "how dangerous is this exact moment?" Our models estimated this for each fire.

LIKELY EXAM / PRESENTATION QUESTIONS

Technical Questions — With Answers

These are the questions a data science instructor is most likely to ask during a class presentation. Read the answers, understand them, then try to explain them in your own words.

> **Q: Why did you use survival analysis instead of a standard classifier?**
>
> Two reasons. First, our outcome isn't just yes/no — it's time-sensitive (12, 24, 48, or 72 hours). Survival analysis is built for time-to-event prediction, not just binary outcomes. Second, about 69% of our fires were contained before reaching the zone. A standard classifier treats those as clean negatives, but survival analysis can actually use the timing of those non-events as meaningful information through right-censoring. We extracted more signal from the same small dataset.

> **Q: What is right-censored data? Why does it matter?**
>
> Right-censored means we know an event hadn't happened up to a certain time, but we don't know what would have happened after. A fire contained at hour 18 is censored at 18 hours — we know it didn't reach the zone by hour 18, but we can't know what it would have done at hour 25. Survival analysis formally accounts for this. Discarding these records would have cut our training data nearly in half and introduced selection bias.

> **Q: How did you handle having only 221 training examples?**
>
> Three ways. First, we chose model types (RSF and GBSA) that are known to generalize well on small datasets rather than models that require large amounts of data to work. Second, we engineered physics-based features that compressed domain knowledge into the inputs, reducing the amount the model had to "figure out" on its own. Third, we used Stratified K-Fold validation to squeeze honest performance estimates out of the limited data we had.

> **Q: What is Stratified K-Fold? Why stratified specifically?**
>
> K-Fold splits data into K groups, trains on K-1, tests on the remaining one, and rotates K times. Stratified means each fold is guaranteed to have the same class ratio as the full dataset — in our case, roughly 31% of fires reached the zone. Without stratification, some folds might have almost no positive examples, making the test results misleading. With only 221 records, that risk was real.

> **Q: Why combine two models? Why not just use the better one?**
>
> RSF and GBSA make different types of errors. RSF is strong at capturing complex nonlinear relationships; GBSA is strong at correcting systematic residual errors through boosting. When their errors aren't correlated, combining them reduces overall variance. The ensemble consistently outperformed either model individually — which is the standard justification for ensembling in competitive data science.

> **Q: What does the concordance index actually measure?**
>
> It measures rank ordering accuracy. Take any two fires — one that reached the zone at hour 20, one at hour 50. The C-index asks: did the model correctly predict that the first fire would arrive sooner? A score of 0.5 means random guessing. Our 0.9737 means the model ranked pairs correctly 97.37% of the time.

Q: Why engineer features rather than use the raw sensor columns?

Raw sensor columns — wind speed, temperature, humidity, coordinates — are measured in different units, on different scales, and their interactions matter more than their individual values. By engineering the danger index (combining wind, heat, and dryness into one score), trajectory alignment (directional relationship between fire and zone), and time-to-impact (physics-based ETA), we gave the model information in a form directly relevant to the prediction task. The engineered features consistently outperformed the raw columns.

Q: How does your leaderboard score compare to other teams?

We observed unusually high scores from some leaderboard participants that were difficult to reconcile with the realistic signal available in a 221-record dataset. This is a known issue in competitions: teams can overfit to the public test set by submitting many times and selecting only their best results. Our 0.9737 is an out-of-fold cross-validation result, which is a more honest measure because the model never saw the test data during training.

Q: What are monotonic probability bounds and why did you apply them?

Our model outputs probabilities of a fire reaching the zone within 12, 24, 48, and 72 hours. Logically, P(reach by 48h) must be ≥ P(reach by 24h) — a longer window can only increase or maintain the probability. Without enforcing this constraint, raw model outputs sometimes violated this ordering due to noise. We post-processed predictions to enforce the monotonic relationship, making the outputs physically interpretable.

Q: What would you do differently if you had more data?

With more records, we could explore deep learning approaches — specifically recurrent neural networks that capture fire progression over time as a sequence. We could also perform more aggressive feature selection and validate across different geographic regions to test generalizability. The current approach was deliberately conservative given the dataset size.

NON-TECHNICAL QUESTIONS

Questions Anyone Might Ask — Including Analogies

These are the softer questions — the ones about process, decisions, and meaning. The professor in this class loves analogies. Use them freely.

Q: Can you explain this project to someone with no data science background?

We built a system that looks at a wildfire in progress — how fast it's moving, what direction it's headed, what the weather is doing — and estimates how likely it is to reach a community in the next 12, 24, 48, or 72 hours. Think of it like a weather forecast, except instead of "70% chance of rain," it's "85% chance this fire reaches your neighborhood within 24 hours." Emergency managers could use that kind of output to decide when to issue evacuation orders.

Q: Why not just use a simple model? Why make it complicated?

We tried to use the simplest model that could actually handle the problem honestly. A simple yes/no classifier would have thrown away information about fires that were contained — which was 69% of our data. That's not simple, that's wasteful. Survival analysis was the right tool because it matched the actual structure of the problem. Complexity for its own sake is bad data science; using the right tool for the right job is good data science.

Q: What was the hardest part of this project?

The hardest part was the dataset size. 221 records is genuinely small for machine learning. Every decision — model choice, feature engineering, validation strategy — had to account for the risk of the model memorizing the training data instead of learning anything real. That required being more deliberate and principled than a larger dataset would have demanded.

Q: How do you know your model is actually good and not just lucky?

That's exactly why we built Stratified K-Fold validation. We tested the model on data it had never seen during training, five separate times, with different test sets each time. The 0.9737 score is the average across those five honest tests. If the model were just getting lucky, that average would be unstable and much lower. Consistency across folds is evidence of genuine learning.

Q: Why does the direction of the fire matter? Explain with an analogy.

Imagine two cars driving toward a town. Car A is going 80 mph but heading slightly away from town. Car B is going 40 mph but aimed directly at it. Which one gets there first? Direction matters as much as speed. Our trajectory alignment feature captured exactly this for wildfires — whether the fire's movement vector was pointed at the evacuation zone or veering away from it.

Q: What's the real-world use case for this model?

Emergency management agencies currently rely on fire behavior models that require manual expert input and can take hours to update. A model like ours — trained on historical trajectories, updated with live sensor data — could provide near-real-time risk estimates to support evacuation decision-making. The 12/24/48/72-hour prediction window maps directly to the planning horizons emergency managers actually work with.

Q: What does it mean that your model "understands fire physics"?

It doesn't understand anything in a human sense. What it means is that the features we fed into the model were built from fire science principles — the danger index combines the known drivers of fire spread (wind, heat, dryness), trajectory alignment uses the geometric relationship between fire movement and the target zone, and time-to-impact is a physics formula, not a pattern the model had to guess. When the input data is grounded in physics, the model's outputs are more interpretable and more likely to generalize.

Q: Could this model be used in production today?

Not directly — 221 training examples is too few for a production system making real evacuation decisions. But it's a valid proof of concept demonstrating that survival analysis is an appropriate framework for this problem. A production system would need years of fire trajectory data across diverse geographic regions, tighter integration with live sensor feeds, and validation by fire behavior domain experts. The approach is sound; the scale isn't there yet.

QUICK REFERENCE

Numbers and Facts to Have Ready

KEY NUMBERS

221 — fire trajectories in our training dataset

31% — fires that actually reached the evacuation zone

69% — fires that were contained (right-censored observations)

0.9737 — our out-of-fold concordance index score

5 — number of folds in our cross-validation

2 — models in our ensemble (RSF + GBSA)

3 — key engineered features (danger index, trajectory alignment, time-to-impact)

4 — prediction horizons (12, 24, 48, 72 hours)

THE THREE THINGS THAT MADE THIS WORK

1. Right model type: Survival analysis for time-to-event data with censoring — not a standard classifier.

2. Physics-grounded features: Engineered inputs based on actual fire science rather than raw sensor columns.

3. Honest validation: Stratified K-Fold cross-validation, not leaderboard score chasing.

ONE ANALOGY TO RULE THEM ALL (IF YOU CAN ONLY REMEMBER ONE)

A standard classifier is a doctor who looks at a patient and says "sick or not sick." Survival analysis is a doctor who says "this patient has a 40% chance of needing surgery within the next 48 hours — and even the patients who recovered on their own told us something about what treatment works." That's the difference. Our wildfire model was the second doctor.